Approval-Based Elections in the 1D Euclidean Domain

Jan Sznajd

Supervisor: prof. dr hab. inż. Piotr Faliszewski

Institute of Computer Science
AGH University of Science and Technology

This dissertation is submitted for the degree of
Master of Science

June 2021

Contents

1 Introduction **1**

1.1 Preference Restrictions 2

1.2 Contributions 3

2 Preliminaries **4**

2.1 Basic Definitions 4

2.2 Preference Restrictions 5

2.2.1 Euclidean Preference Restrictions 6

2.3 Methodology/Techniques (WIP: Inny tytuł?) 8

2.3.1 Integer Linear Program 8

3 Voter/Candidate Range Profile Detection **11**

3.1 Profile Structure Detection 11

3.2 Integer Linear Program 12

3.3 WIP: Consecutive Ones Detection — ILP vs SAT 14

4 Voter/Candidate Range Profiles **15**

4.1 Domain analysis 15

4.1.1 Experiment 1: How many profiles are TVCR or VCR? 15

4.1.2 Experiment 2: How many profiles are TVCR, CR, VR or FCOP? . 16

4.2 Transforming a TVCR Profile into a CR or a VR One 23

4.2.1 Experiment 3: How different is a TVCR profile? 23

5 Control Problem **30**

5.1 Overview 30

5.1.1 AV-CC-DV for Single Peaked (VR = CI = row C1P = $r_c = 0$) . . . 30

5.1.2 AV-CC-DV for Candidate Range = VI = column C1P = $r_v = 0$ . . . 31

5.2 AV-CC-DV for the VCR profiles 31

5.2.1 Sketch Proof - CR case 32

5.2.2 Sketch Proof - VR case 33
5.2.3 Sketch Proof - VCR (not C1P) case 35
5.3 AV-CC-AV for the VCR profiles 38

6 Summary **39**
6.1 Conclusions 39
6.2 Future Work 39

Bibliography **40**

Chapter 1

Introduction

Elections are a ubiquitous element of our society in one form or another. For example, societies need to choose members of parliaments, judges have to pick winners of sports competitions. However, elections are not only applicable to politics or competitions. In more day-to-day examples of elections, teams have to schedule meetings times, or video streaming platforms need to provide recommended movies list to the users. Each of these problems may require a different type of an election and a different strategy of selecting the winner (or winners). The differences come from the underlying properties and objectives of the problem. For example, in presidential elections, we want to choose the candidate supported by the majority of the voters. It is not the case for the parliamentary elections, where the goal is to select members of the parliament in such a way that the opinions of the voters are represented proportionally. Another example would be a meeting scheduling problem, where we will try to pick the time, such that, we maximize the number of participants, even though, the selected time might not be optimal for anyone (e.g., early morning). In sports competitions, especially in the seasonal ones, i.e., the ones consisting of series of events, we need to come up with a strategy of calculating overall ranking at the end of the season, based on the aggregated results from the single events. For example, in FIS Alpine Ski World Cup, or in Formula 1 World Championship, the overall winners achieve very high scores consistently throughout the season, but might not have necessarily achieved the highest scores (1st place) in the majority of the individual events.

Another key distinction between the given examples, is whether a given election has just one winner, or does it lead to selecting multiple candidates. More formally, in computational social choice theory, these election types are called *single-winner* or *multiwinner* elections. Multiwinner (committee) elections, are even more widespread than the single-winner ones, and in most cases they have different properties and require different voting rules. For an overview of various committee election types and their related properties, see works

of Faliszewski et al. (2017) and Elkind et al. (2017). A committee election is the process of choosing a fixed-size subset of candidates (i.e., the committee), based on the voters' preferences. Voters' ballots (preferences) can be represented in various different ways. In this work we focus on approval elections, that is, on elections where each voter provides a subset of candidates that he or she approves of. Another notable type of elections is the ordinal model, where the voters order the candidates from the most to the least preferred ones. For an extensive survey of the approval model and current state of research in this area, regarding committee elections, we point to a survey by Lackner and Skowron (2020).

Winner determination is one of the core problems in computational social choice theory, however not the only one. Other ones include, e.g., control problems (Bartholdi et al. (1992); Hemaspaandra et al. (2009)), which refer to certain ways of manipulating elections. In short, we try to make a given candidate win (or lose) by changing the structure of an election, that is, by adding or deleting voters or candidates. Other interesting problems, that we are not going to explore in this work, include manipulation (Bartholdi et al. (1989); Faliszewski and Procaccia (2010)), bribery (Faliszewski et al. (2006)), justified representation (Aziz and Walsh (2014)) and many others (Brandt et al. (2016)).

1.1 Preference Restrictions

Many of the above-mentioned social choice problems are known to be computationally hard. We can deal with this complexity using various methods, such as designing approximation (random or deterministic) or parametrized algorithms. An alternative approach is to introduce restrictions on the preference domains. In other words, we restrict the structure of voters' ballots. There is a successful and active line of research in this area. This begins with the notion of *single-peaked (SP)* (Black (1958); Arrow (1951)) and *single-crossing (SC)* (Mirrlees (1971); Roberts (1977)). Much more recently, Barberà and Moreno (2011) introduced a concept of the *top-monotonicity*, a generalization of SP and SC restrictions. These models provided a basis for numerous and promising results in circumventing the computational complexity in a variety of problems. (Walsh (2007); Faliszewski et al. (2009); Skowron et al. (2015); Brandt et al. (2015); Elkind et al. (2016))

However, in the context of this work we need to emphasize a survey by Elkind and Lackner (2015). Unlike most of the above-mentioned examples, which considered the ordinal model of preferences, the survey explored well-known preference restrictions in the setting of approval preferences, and introduced several new ones. This includes models such as *Voter Interval (VI)* and *Candidate Interval (CI)*, both based on the *Dichotomous Euclidean (DE)* restriction. Euclidean models are very intuitive, a simplified explanation of such models,

is that, each candidate and each voter is assigned an ideal point in some Euclidean space $\mathbb{R}^d$, this point represents their opinion on some d issues. The points are assigned in such a way, that voters prefer the candidates that are closer to them. Godziszewski et al. (2021) extended this model, making it more versatile, by using a radius in addition to a point, to represent candidates and voters opinions. Godziszewski et al. (2021) named the new model a *Voter/Candidate Range (VCR)* model, and later analyzed the two-dimensional case and briefly the one-dimensional one, for committee election problems.

1.2 Contributions

In this work we focus on the one-dimensional Voter/Candidate Range model and our main contributions are as follows:

1. In Chapter 3, we provide detection algorithm for the 1D-VCR property, based on a reduction to integer linear programming (ILP). Additionally, we apply it to detecting 1D-VR and 1D-CR profiles, later we do a performance comparison with a solution based on a 2-SAT reduction from Fitzsimmons and Lackner (2019).

2. In Chapter 4, we analyze the 1D-VCR domain and compare it to other well known models. We use multiple statistical cultures to generate random elections in the 1D-VCR model, then we check how many of them do not satisfy 1D-VR or 1D-CR properties. Furthermore, we inspect how different are the 1D-VCR elections from the 1D-VR and 1D-CR ones.

3. In Chapter 5, we propose a polynomial time algorithm for a certain control problem under the 1D-VCR model, expanding on the solutions for 1D-CR and 1D-VR models, due to Magiera and Faliszewski (2014) and Faliszewski et al. (2009), respectively.

(WIP: Powyżej, używam VR i CR, mimo że nigdzie o nich nie wspomniałem, czy powinienem napisać że VI=CR, CI=VR?)

We do not discuss the related work right now, and instead we do it throughout this thesis, as the discussion will greatly benefit from the added context and introduced definitions in the subsequent chapters.

Chapter 2

Preliminaries

First, in Section 2.1, we introduce basic definitions and core concepts, together with simple examples. Later, in Section 2.2, we discuss preference restrictions, and how can we leverage them to design better algorithms, for many of the computational social choice problems. Finally, in Section 2.3, we explain some computational methods (tricks) we have used in this work, that we believe are not necessarily straightforward.

2.1 Basic Definitions

For an integer t, by $[t]$ we mean the set $\{1,...,t\}$. To describe a matrix $\mathbf{A}$ with r rows and c columns, we use $\mathbf{A}_{r\times c}$. By $\mathbf{A}[i,j]$ we refer to the entry in i-th row and in j-th column of a matrix $\mathbf{A}$.

Approval Elections

An approval-based election (in short, an *election*, or interchangeably a *profile*) is a pair $E = (C,V)$, where $C = \{c_1,...,c_M\}$ is a set of candidates and $V = \{v_1,...,v_N\}$ is a set of voters. We use M and N for the number of candidates and voters respectively. For each voter $v \in V$, his or hers preferences are represented as a set of approved candidates $A(v)$, we call it an *approval set*. Furthermore, for each candidate $c \in C$, we define $V(c) = \{v \in V \mid c \in A(v)\}$, a set of voters who approve candidate c.

In practice, we often use an additional, simplified notation of representing an election. We redefine A as a binary matrix (a matrix, where each entry is in $\{0,1\}$), with N rows and M columns. For $i \in [N]$ and $j \in [M]$, entry $A[i,j]$ is 1 if voter v_i approves of candidate c_j, and 0 otherwise. In other words, each row of the matrix is a vector representation of a given voter's

approval set. We refer to it as an *approval matrix* or a *preference matrix*. It will always be clear from the context which notation we are using.

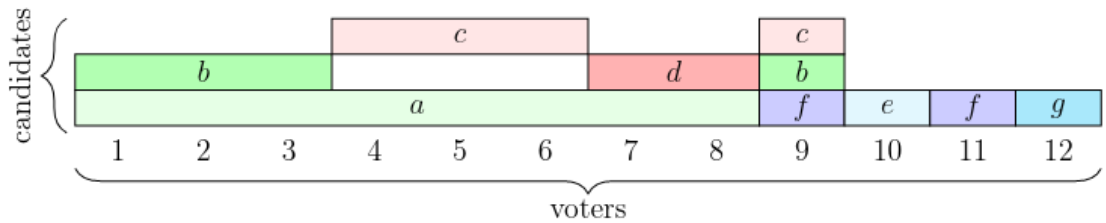


Figure 2.1 Example of an approval election. A candidate is approved by a voter if his or hers block is above the voter's point. Temporary image from Lackner and Skowron (2020)

Example 1. *(WIP: Opis wyborów z figury) Following the example from Figure 2.1 ...*

Multiwinner Committee Selection Rules

Voting rules are algorithms used to, select a committee (winners) — fixed size subset of the candidates, based on the voters preferences, for a given election. Each rule has different properties and must be selected according to the goal of the election. As this is not the main focus of this work we point to Faliszewski et al. (2017), for a detailed overview of multiwinner voting rules.

Nonetheless, we introduce the most basic voting rule for the approval model, simply called *Approval Voting (AV)*, introduced by Brams and Fishburn (1983).

Definition 2.1.1 (Multi-Winner Approval Voting (AV))**.** *For a given election $E = (C, V)$ and a size of committee k, AV rule selects the k candidates with the highest approval scores. AV-score of a given candidate c, is the number of voters who approve of c, and formally, is defined as $score_{AV}(E, c) = |\{v \in V : c \in A(v)\}|$.*

Example 2. *(WIP: Opis wyborów z poprzedniego przykładu + kto wygra) Continuing with the election from Example 1*

2.2 Preference Restrictions

As a reminder from Section 1.1, many computational social choice problems such as winner determination, or control problems are known to be computationally hard. Preference restrictions, i.e., restrictions on the structure of voters' ballots, are a very effective solution to circumvent the computational complexity of these problems.

2.2.1 Euclidean Preference Restrictions

Euclidean preference restrictions, in the approval voting model, are foundation of this work. All the problems we solve in the subsequent chapters are structured around this concept. Unless stated otherwise, throughout this work, we consider one-dimensional variants of the Euclidean models.

Let us introduce restrictions that we explore in this work. Starting with the most basic one, that is a predecessor to the other ones, in this family of restrictions.

Restriction 1 (Dichotomous Euclidean (DE))**.** *[Elkind and Lackner (2015)]*

Given an election $E = (C,V)$, we say that E satisfies DE if there is a mapping ρ of voters and candidates into the real line such that for every voter $v \in V$ there exists a radius r_v, and it holds that $v = \{c : |\rho(v) - \rho(c)| \leq r_v\}$. (WIP: Czy w ogóle definiować DE?, czy jest potrzebna cytacja?, ta definicja nie jest specjalnie spójna z pozostałymi oznaczeniami / nie oczywista)

The DE preference restriction is somewhat limited, it permits for only one agent type, i.e., candidate or voter, to have a radius parameter, while the other one is represented just by his or hers point. We consider the DE model in one of the two variants:

1. The voter-range model, where voters have the radius parameter. In other words, if a voter approves a candidate then he or she also approves all the closer ones.

2. The candidate-range model, where candidates have the radius parameter. We say that if a candidate is approved by a voter, then he or she is also approved by all the voters closer to him or her.

Below we provide a formal definition of these two models, as well as their generalization which allows all agents to have a radius parameter.

Restriction 2 (Voter/Candidate Range (VCR))**.** *[Godziszewski et al. (2021)]*

Given an election $E = (C,V)$, we say that the voters have one-dimensional Euclidean preferences if for each agent $a \in C \cup V$ (i.e., for each candidate and each voter) there exists a point $x_a \in \mathbb{R}$ and nonnegative real value $r_a \in \mathbb{R}$ such that:

$$c \in A(v) \iff |x_c - x_v| \leq r_c + r_v$$

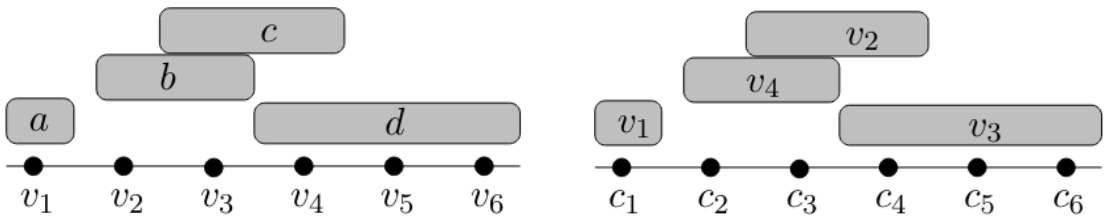
We can think of the VCR restriction as for an agent $a \in C \cup V$ the point x_a describes a's ideal position in $\mathbb{R}$, representing his or hers opinion. For a candidate $c \in C$, radius r_c can be seen as c's charisma, or in other words their ability to reach a number of different voters

(e.g., a centrist candidate possibly would have a larger reach, than an extremist one). For a voter $v \in V$, radius r_v corresponds to v's willingness to compromise, that is, how different (faraway) opinion they would still accept from their ideal one. Moreover, there are two special cases of the VCR model.

Restriction 3 (Candidate Range (CR))**.** *We say that a VCR election $E = (C,V)$, is CR if for each voter $v \in V$, their radius r_v is equal to zero.*

Restriction 4 (Voter Range (VR))**.** *We say that a VCR election $E = (C,V)$, is VR if for each candidate $c \in C$, their radius r_c is equal to zero.*

The CR and the VR models are equivalent to the voter interval (VI) and the candidate interval (CI) ones, both introduced by Elkind and Lackner (2015). Godziszewski et al. (2021) simply use a different terminology.



(a) CR election (b) VR election

Figure 2.2 Example of elections with the CR and the VR preference restrictions. Temporary Image from Elkind and Lackner (2015)

The VCR domain is larger than the CR and the VR domains combined. Below in Example 3, we show an election that is VCR, but neither CR nor VR. We explore this topic further in Chapter 4.

Example 3. *Given an election E with the candidates $C = \{a,b,c,d\}$ and the voter's approval sets $\{a,b\}$, $\{b,c\}$, $\{b,d\}$, and $\{a,b,c,d\}$.*

To see why, ... (WIP: Podać punkty i promienie (VCR), następnie opisać dlaczego nie jest CR ani VR ... Obrazek?)

2.3 Methodology/Techniques (WIP: Inny tytuł?)

2.3.1 Integer Linear Program

Integer linear programming (ILP) is a common approach to solve NP-hard problems, furthermore it can be used successfully for other problems as well. To solve a problem by reduction to the linear program, first we need to define the given problem as a system of linear inequalities and an objective function. Formally, for $j \in [n]$, a single inequality consists of a set of decision variables x_j, a set of parameters a_j and a constraint b, together giving a formula: $a_1x_1 + a_2x_2 + ... + a_nx_n \leq b$ However, in practice there are multiple constraint inequalities, and we use more general notation to represent an i-th inequality:

$$\sum_{j=1}^{n} a_{ij}x_j \leq b_i$$

Besides constraints, we define an objective function with parameters c_j:

$$\sum_{j=1}^{n} c_jx_j$$

Now, having defined inequalities and the objective, the goal is to find a vector of values for the x_j, such that it maximizes (or minimizes) the value of the objective function, while keeping all the inequalities true. In general, all the variables of a linear program can be fractional, and such program can be solved in polynomial time. However, there is a special case called *(mixed) integer linear program*, that is, when we require some variables to be integers. Solving integer linear programs is NP-hard. Despite this, there are available state-of-the-art linear program solvers, that solve ILPs effectively. In this work we use one of them, the Gurobi solver. For the complete theory of (integer) linear programming, we point to the book by Wolsey and Nemhauser (2014).

Not every problem can be directly defined as an integer linear program. However, we can apply basic techniques and tricks to make the problem suitable. For example, this is the case for our solution to the VCR profile detection problem described in Chapter 3. Below we present these methods:

Big-M Method. This technique is used to overcome a variety of problems, such as representing absolute or minimum/maximum values as linear inequalities. Another examples would be defining a logical conditions (And/Or) between the inequalities or using indicators

(i.e., if-then logic). These problems are not linear in their nature. However, by taking certain steps we can linearize them, and then use them in a linear program.

Having a set of m linear inequalities with n variables represented as:

$$\sum_{j=1}^{n} a_{ij}x_j \leq b_i \qquad i \in [m], j \in [n]$$

The fundamental concept of the BigM method is to introduce to the inequalities, an additional large constant coefficient M (hence Big), which is greater than the maximum value of any variable x_j. Based on how we do it decides which of the aforementioned problems we solve. Below in Example 4, we showcase the BigM technique to represent *OR* condition between inequalities.

Example 4. *We have two inequalities and want to represent a logical OR condition between them.*

$$\sum_{j=1}^{n} a_{1j}x_j \leq b_1 \quad or \quad \sum_{j=1}^{n} a_{2j}x_j \leq b_2.$$

To do so, we introduce a large constant M and two binary variables $y_1, y_2 \in \{0,1\}$. First we transform the inequalities:

$$\sum_{j=1}^{n} a_{1j}x_j \leq b_1 + M(1-y_1)$$

$$\sum_{j=1}^{n} a_{2j}x_j \leq b_2 + M(1-y_2)$$

Then, we add one more constraint, binding the inequalities together:

$$y_1 + y_2 \geq 1$$

This constraint enforces that at least one of the inequalities will be true.

The BigM method is a valuable and widely used tool, however it has some downsides, mostly related to the numerical issues. That is why, most ILP solvers provide additional built-in mechanisms for applying it. For further reading, we point to the reference manual of Gurobi Optimization (2021). We use one of those mechanisms for our linear program in Chapter 3, namely an *indicator constraint*. It allows one to create constraints representing if-then logic. Formally, we have to introduce a binary variable $z \in \{0,1\}$, and then we can define an indicator constraint in the form of:

$$z = f \longrightarrow \sum_{j=1}^{n} a_{ij}x_j \leq b_i$$

The constraint states that if variable z is equal to $f \in \{0,1\}$, then the linear constraint should hold. On the other hand, if $z = 1 - f$, then the linear consist may be violated.

Chapter 3

Voter/Candidate Range Profile Detection

First, in Section 3.1, we discuss various approaches to preference recognition, and we present selection of related work. Followed by Section 3.2, we describe our main contribution of this chapter. That is, an algorithm based on a reduction to ILP for deciding if a given approval profile has the VCR structure. Finally, in Section 3.3 we experimentally compare the performance of our ILP based solution with the one based on a reduction to 2-SAT, for a task of recognizing the CR and the VR preference restrictions.

3.1 Profile Structure Detection

To take advantage of the restrictions defined in Section 2.2, we need to design detection algorithms. They can be used to check whether given election profiles satisfy particular structural properties.

NP-Hard for $d \geq 2$ from Peters (2017) Other examples: SPOC Peters and Lackner (2020), top-monotonicity Magiera and Faliszewski (2019), spsc Elkind et al. (2020).

For example, to check the VR (CR) property it is enough to check for the *consecutive ones property* in the preference matrix.

Definition 3.1.1. *A binary matrix (i.e., a matrix consisting of zeros and ones) has the consecutive ones property (C1P) if its columns be permuted in such a way that for each row the ones are consecutive, i.e., the 1s form an interval.*

In this work we refer to the term *C1P* more broadly, using it interchangeably, for an analogous scenario where we permute the rows and look for the consecutive 1s in every column in a given matrix. Booth and Lueker (1976) provided a linear-time C1P detection algorithm based on the PQ-Tree data structure.

Proposition 1 (Elkind and Lackner (2015))**.** *Deciding whether an approval profile satisfies VR or CR is possible in $O(m \cdot n)$ time.*

Proof. Let $C = \{c_1, ..., c_m\}$ and $V = \{v_1, ..., v_n\}$, we create a $n \times m$ matrix A (a_{ij}): for each $i \in [n]$ and $j \in [m]$: $a_{ij} = 1$ if $c_j \in A(v_i)$ and 0 otherwise. We can see that for the VR property, permuting the columns of matrix A so that ones form an interval in each row is equivalent to permuting candidates so that the set of candidates approved by each voter forms an interval. For the CR case, we just need to transpose the matrix created for the VR. □

Our focus lies on the one dimensional VCR property. Having preference matrix as input we have to provide for each agent (voter/candidate) a point on the real line and a nonnegative real value representing radius, so that these parameters satisfy the VCR property.

3.2 Integer Linear Program

An algorithm based on ILP solving is an excellent choice for the first attempt at designing a detection algorithm. It provides an efficient and relatively simple solution. Furthermore, it can be used as a fast benchmark for validating future solutions. Below we provide an ILP for the VCR profile recognition, together with additional preliminary steps.

Transforming the Definition of VCR into Linear Constraints

Before formulating an ILP for detecting VCR profiles, we have to redefine the definition of VCR in order to make it appropriate for an ILP solver. The definition is based on the euclidean distance which cannot be directly represented in an ILP. Equation 3.2 shows transformation of the VCR definition into linear inequalities.

As a short reminder from Section 2, for the candidates C and voters V, 1D-VCR is defined as:

$$\forall\, c \in C\, \forall\, v \in V : c \in A(v) \Leftrightarrow |x_c - x_v| \leq r_c + r_v \tag{3.1}$$

Where x_a and r_a represent agent's point on the real line and their radius respectively.

(WIP: W jaki sposób mogę zwięźle, bez ponownego wpropadzania wszystkich oznaczeń, przypomieć definicję? O ile w ogóle jest to potrzebne...)

$$\begin{aligned} &|x_c - x_v| \leq r_c + r_v \\ &\iff -(r_c + r_v) \leq x_c - x_v \leq r_c + r_v \\ &\iff x_c - x_v \leq r_c + r_v \;\; \vee \;\; -(x_c - x_v) \leq r_c + r_v \end{aligned} \tag{3.2}$$

(WIP: Poprawić formatowanie, jak?)

The constraint obtained from the equation 3.2 is not suitable yet for an ILP because it contains logical *OR* condition which again cannot be directly represented in an ILP. To transform it into linear form, we apply commonly called *BigM* technique, as shown below.

$$\begin{aligned} x_c - x_v &\leq r_c + r_v + M(1 - z_1) \\ -(x_c - x_v) &\leq r_c + r_v + M(1 - z_2) \\ z_1 + z_2 &\geq 1 \\ z_1, z_2 &\in \{0, 1\} \end{aligned} \tag{3.3}$$

First, we have to introduce a large constant M (larger than $max(|x|)$) and two binary variables z_1 and z_2. Now if any of the z_i is 0 then the corresponding inequality will be true, and the additional constraint ($z_1 + z_2 \geq 1$) guarantees that at least one of the z_i will be 1.

(WIP: Trochę namieszane wytłumaczenie, bo tak naprawdę problem z *OR* jest tylko dla wariantu if $c \notin A(v)$, czyli $x_c - x_v + M(1 - z) \geq r_c + r_v + 1$ ...)

The BigM technique is common enough, for most solvers to provide additional built-in mechanisms for applying it. Using them is usually recommended, as it allows a solver for better internal optimization. In Gurobi solver we use *indicator constraint*, which allows one to create versatile constraints in form of: $z = f \rightarrow a^T x \leq b$. The constraint states that if the binary indicator variable z is equal to f, where $f \in \{0, 1\}$, then the linear constraint $a^T x \leq b$, where a and b are data coefficients vectors and x represents the decision variables, should hold. On the other hand, if $z = 1 - f$, the linear constraint may be violated.

(WIP: BigM jest używane do reprezentacji różnych 'funkcji', nie tylko *OR*, np. *max*, *abs*, etc., czy też o tym wspomnieć?)

Detection Algorithm

Final formulation of the ILP for recognising VCR property in a given election profile is shown in Figure 3.4. The program accepts preference matrix A as input, and if solution exists it outputs every agent's position x_a and radius r_a. Additionally, we use two binary variables p and q, required by the indicator constraints.

$$
\begin{aligned}
&\textbf{check feasibility}: \\
&\quad \textbf{subject to}: \\
&\qquad x_a \in \mathbb{R} && \forall\, \mathrm{a} \in C \cup V \\
&\qquad r_a \in \mathbb{R}^{0+} && \forall\, \mathrm{a} \in C \cup V \\
&\qquad p_{cv} \in \{0,1\} && \forall\, \mathrm{c} \in \mathrm{C}\, \forall\, \mathrm{v} \in V \\
&\qquad q_{cv} \in \{0,1\} && \forall\, \mathrm{c} \in \mathrm{C}\, \forall\, \mathrm{v} \in V \\
&\quad \text{if } c \in A(v): \\
&\qquad x_c - x_v \leq r_c + r_v \\
&\qquad -(x_c - x_v) \leq r_c + r_v \\
&\quad \text{if } c \notin A(v): \\
&\qquad p_{cv} == 1 \rightarrow x_c - x_v \geq r_c + r_v + 1 \\
&\qquad q_{cv} == 1 \rightarrow -(x_c - x_v) \geq r_c + r_v + 1 \\
&\qquad p_{cv} + q_{cv} >= 1
\end{aligned}
\tag{3.4}
$$

3.3 WIP: Consecutive Ones Detection — ILP vs SAT

Chapter 4

Voter/Candidate Range Profiles

In this chapter we explore the Voter/Candidate Range domain, describe some of its properties, and compare it with the Voter Range and Candidate Range ones. We already know that the VCR domain is larger than the VR and CR ones combined (Section 2, Example 3). However, we would like to know how much bigger it actually is, and if we can significantly benefit from further development of algorithms based on the VCR model.

4.1 Domain analysis

We have performed several experiments, in each of them we check how often a profile has the TVCR restriction, compared to the other ones. For the first experiment we focus on the small size elections and only observe the overview of the properties. For the second one, we do a more thorough examination. We check larger profiles, and compare the properties in detail. In every experiment we use the detection algorithm described in Section 3.2.

For the sake of text clarity and more explicit naming, we introduce additional notations to describe some special cases of the VCR preference restriction.

Definition 4.1.1. *A profile P is **Truly VCR (TVCR)**, if P satisfies the VCR property and does not satisfy either of the VR and the CR properties.*

Definition 4.1.2. *A profile P is **Fully COP (FCOP)**, if P satisfies both the VR and the CR properties (see Definition 3.1.1, for a reminder of the COP).*

4.1.1 Experiment 1: How many profiles are TVCR or VCR?

For this experiment we performed a complete search of the domain. That is, we generated every possible profile of a given size and checked if it has the VCR property,

and if so we checked again if it was the TVCR variant. We focused on the small size of elections, having elections of M candidates and N voters, we considered $(M,N) \in \{(3,3),(4,4),(5,5),(4,6),(6,4)\}$.

The results are presented in Figure 4.1. They show that the VCR domain is relatively large. However, for these small election sizes, only a limited number of profiles are Truly VCR (i.e., neither VR nor CR). For example, looking at the two smallest elections, for the 3 candidates, 3 voters case, there is no TVCR profiles at all. For the 4 by 4 case, there is only 96 of them, out of 2^{16} profiles. Another observation is a decrease in a number of the VCR profiles, in favor of the not VCR ones, with the increase of the election size (e.g., for the 5 candidates, 5 voters case, only about 60% of profiles are VCR).

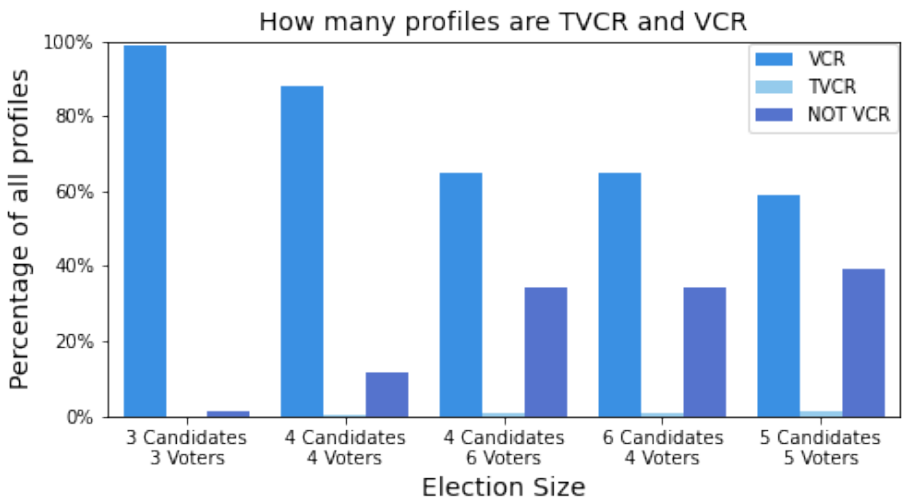


Figure 4.1 Relative number of the profiles with a given property (VCR and TVCR), out of all possible ones for a given election size, from Experiment 1

4.1.2 Experiment 2: How many profiles are TVCR, CR, VR or FCOP?

In this experiment, we only consider the VCR profiles. We want to find out, how their numbers are distributed across different variants of the VCR restriction. We generated a suitable number of random VCR profiles and checked how many of them are neither CR nor VR (TVCR). We grouped the results into three additional categories, besides the TVCR one, that is, VR and CR (FCOP), VR and not CR, CR and not VR.

We employ several strategies to generate a large number of profiles with distinct characteristics. We use various statistical cultures (distributions) to pick random points and radii representing the candidates and voters. Furthermore, we choose the random distribution independently for the position and radius parameters, and independently for the candidates

and voters. We describe these categories in detail below. For each category pair (in total, 20 pairs) we generate 1000 elections with 20 candidates and 20 voters. Then we repeated the experiment for the elections consisting of 40 candidates and 40 voters.

Position Parameter

We consider four ways of generating the points, representing the positions of the candidates and the voters:

1. The *Uniform Candidate Positions, Uniform Voter Positions (UCP/UVP)* model, where the points for both, the candidates and the voters are selected uniformly at random from the $[-10, 10]$ interval.

2. The *Asymmetric Gauss Candidate Positions, Asymmetric Gauss Voter Positions (GCP/GVP)* model, where we take 70% of points from a Gaussian distribution with mean value -3 and standard deviation 1.8, and the remaining 30% come from a Gaussian with mean 3 and the same standard deviation. This model is inspired by the 2 dimensional model from the work of Godziszewski et al. (2021). The goal is to simulate a society with a large majority, and a minority with the opposite opinion.

3. The *Uniform Candidate Positions, Asymmetric Gauss Voter Positions (UCP/GVP)* model, it is a combination of the previous two models, where the candidates' points are selected uniformly at random, and for the voters we use the asymmetric Gaussian model. We use the same parameters for both of the distributions as above.

4. The *Asymmetric Gauss Candidate Positions, Uniform Voter Positions (GCP/UVP)* model, is a reversed version of the previous model, with the distributions swapped.

Radius Parameter

In this category we consider five ways of generating the radii for the candidates and the voters. We use the following models:

1. The *Gauss Candidate Radii, Gauss Voter Radii (GCR/GVR)* model, where we choose the radii from the Gaussian distribution with mean value 1.5 and standard deviation 0.5. We use the same distribution for the candidates and the voters.

2. The *Small Uniform Candidate Radii, Gauss Voter Radii (SUCR/GVR)* model. We pick the radii for the candidates uniformly at random from the interval $[0.7, 1.2]$, and for the voters we use the same Gaussian as in the previous model (with mean 1.5 and standard deviation 0.5).

3. The *Gauss Candidate Radii, Small Uniform Voter Radii (GCR/SUVR)* model. In this model we swap the distributions used in the previous one, that is, the radii for the candidates are picked from the Gaussian and the radii for the voters from the uniform distributions.

4. The *Small Uniform Candidate Radii, Small Uniform Voter Radii (SUCR/SUVR)* model. We pick the radii for both the candidates and the voters uniformly at random from the interval $[0.7, 1.2]$.

5. The *Large Uniform Candidate Radii, Large Uniform Voter Radii (LUCR/LUVR)* model. We pick the radii for both the candidates and the voters uniformly at random from the interval $[0, 3]$.

At first, it might be surprising why we do not include a constant radius as one of the distributions. We disregard this possibility because, then, all the profiles would be VR and CR (FCOP). Below in Proposition 2, we show why.

Proposition 2. *If every agent (candidate/voter) in a VCR profile has the same constant radius, then this profile satisfies both the VR and the CR properties (it is FCOP).*

Proof. Consider a VCR election $E = (C, V)$, where $C = \{c_1, ..., c_M\}$ and $V = \{v_1, ..., v_N\}$, and a constant radius R. An agent $a \in C \cup V$ is represented by a point x_a and a radius r_a, and in this special case every agent has the same radius $r_a = R$. We construct a CR election $E' = (C', V')$, where each voter $v' \in V'$ has radius $r_{v'} = 0$ and each candidate $c' \in C'$ has radius $r_{c'} = 2R$. We will show that these elections are identical. From the VCR definition, we know that for each candidate $c \in C$, if c is approved by a voter v_i, then

$$|x_c - x_{v_i}| \leq r_c + r_{v_i} \iff$$
$$|x_c - x_{v_i}| \leq R + R$$

Similarly, for each candidate $c' \in C'$ if c' is approved by a voter v'_i, then

$$|x_{c'} - x_{v'_i}| \leq 2R + 0$$

We showed that the elections E and E' are equivalent. Analogous reasoning works for the VR case, where each candidate has radius zero, completing the proof. □

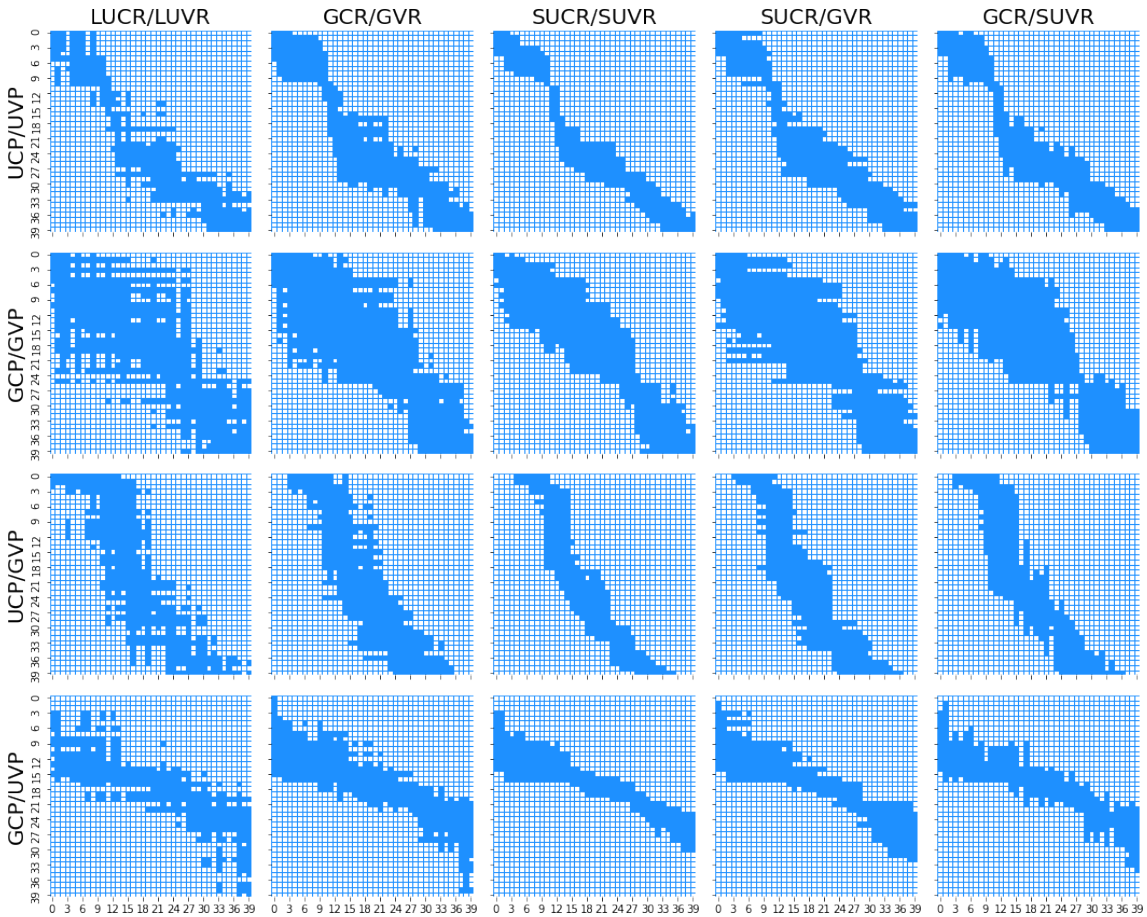


Figure 4.2 Examples of approval matrices for the elections of 40 candidates and 40 voters, from Experiment 2. Each subplot is a visual representation of an approval matrix of a selected profile. Solid color corresponds to 1 and white to 0 in cells of the matrix. To create a grid, we use radius candidate/voter models for the columns, and position candidate/voter models for the rows, e.g., top-left subplot is visualization of a profile generated with the LUCR/LUVR radius model and the UCP/UVP position model.

Results

First, in Figure 4.2, we present the visualization of our example profiles approval matrices and how their visual structure is impacted by the generation parameters. Each subplot represents a different combination of the generation parameters pairs. Columns correspond to the radius candidate/voter models, and rows to the position candidate/voter models.

Before, we further describe the results of the experiment, we need to observe trends in the visualization and explain some basic ideas. Based on the visualizations, we know that the VCR profiles produce semi-structured approval matrices. That is, the ones in an approval matrix create roughly continuous blocks (intervals), they form a cluster. However, in certain

circumstances, these blocks might have some irregularities (i.e., a 0 in sequence of 1's). The perfect example of this phenomenon are profiles from the first column in Figure 4.2, that is, profiles generated using the LUCR/LUVR radius model. We can spot the "holes" in otherwise regular structure of 1's. Another important trend in these visualizations is the general direction in which the block structure is skewed. For profiles in the first and second row of Figure 4.2 (i.e., the UCP/UVP and the GCP/GVP models), the skew is on a diagonal from the top-left to the bottom-right corner. However, profiles from the last two rows (i.e., the UCP/GVP and the GCP/UVP models), while maintaining general diagonal direction, they have a significant shift. The UCP/GVP ones are skewed vertically, opposite to the GCP/UVP ones, which are skewed horizontally.

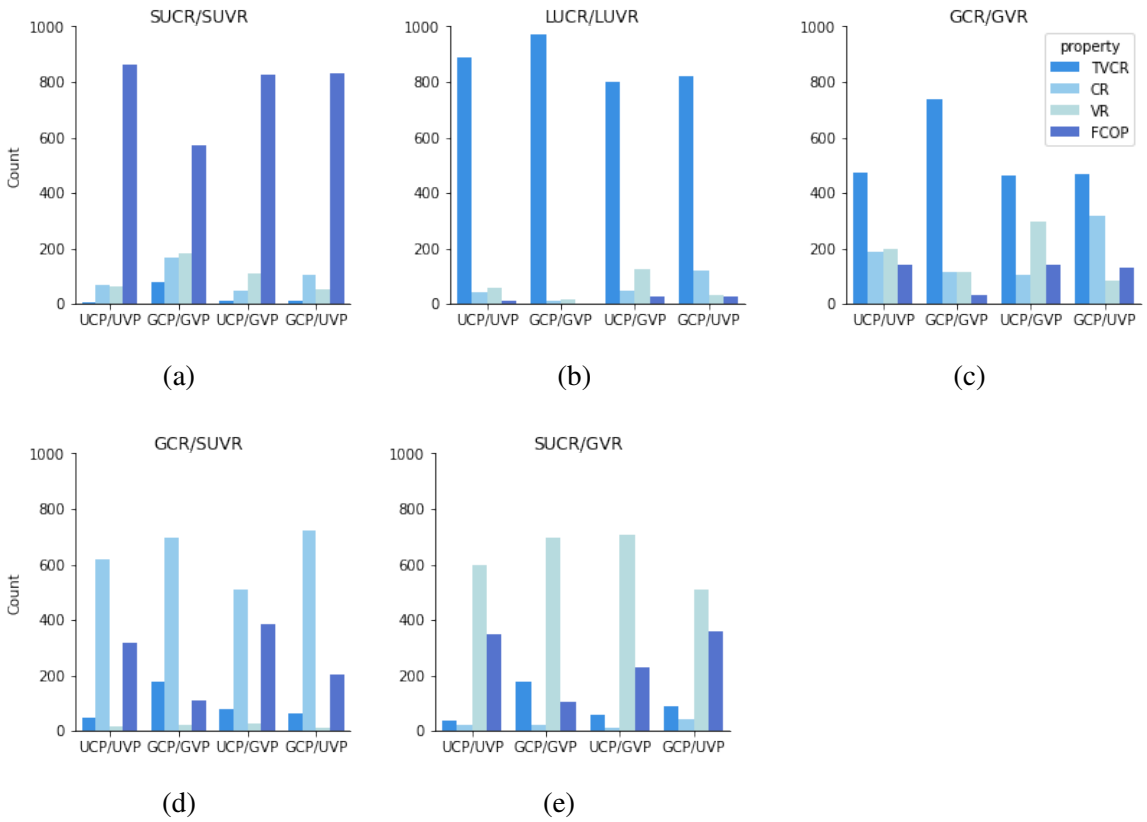


Figure 4.3 Number of profiles with a given property, for the elections of 20 candidates and 20 voters, from Experiment 2. Results are grouped by the parameters used for the profile generation. Each subplot corresponds to the selected radius candidate/voter model, and position candidate/voter models are represented on the x-axis.

In the following discussion, we tie these observations with the aggregated results of the experiment visualized in Figure 4.3 and Figure 4.4. In Figure 4.3 we show how many of the generated profiles have a given preference restriction. To understand the results better,

we split the collected data into groups. We use separate subplots to distinguish the radius candidate/voter models, and the position candidate/voter models are presented on the x-axis of each subplot. Figure 4.4 is almost identical, the only difference is the first one shows the results for the 20 candidates and 20 voters elections, whereas the second one describes the larger elections of 40 candidates and 40 voters.

Election Dimensions Our results for two election sizes have one crucial difference. Looking briefly at Figure 4.3 (20 candidates, 20 voters) and Figure 4.4 (40 candidates, 40 voters), we immediately see that in the larger elections the TVCR property is much more frequent. Election sizes in the experiment are relatively small when compared to real world scenarios. We can expect for this trend of increased number of TVCR profiles to continue for even larger elections. This result is very promising, as it strongly encourages further development and research related to the VCR domain.

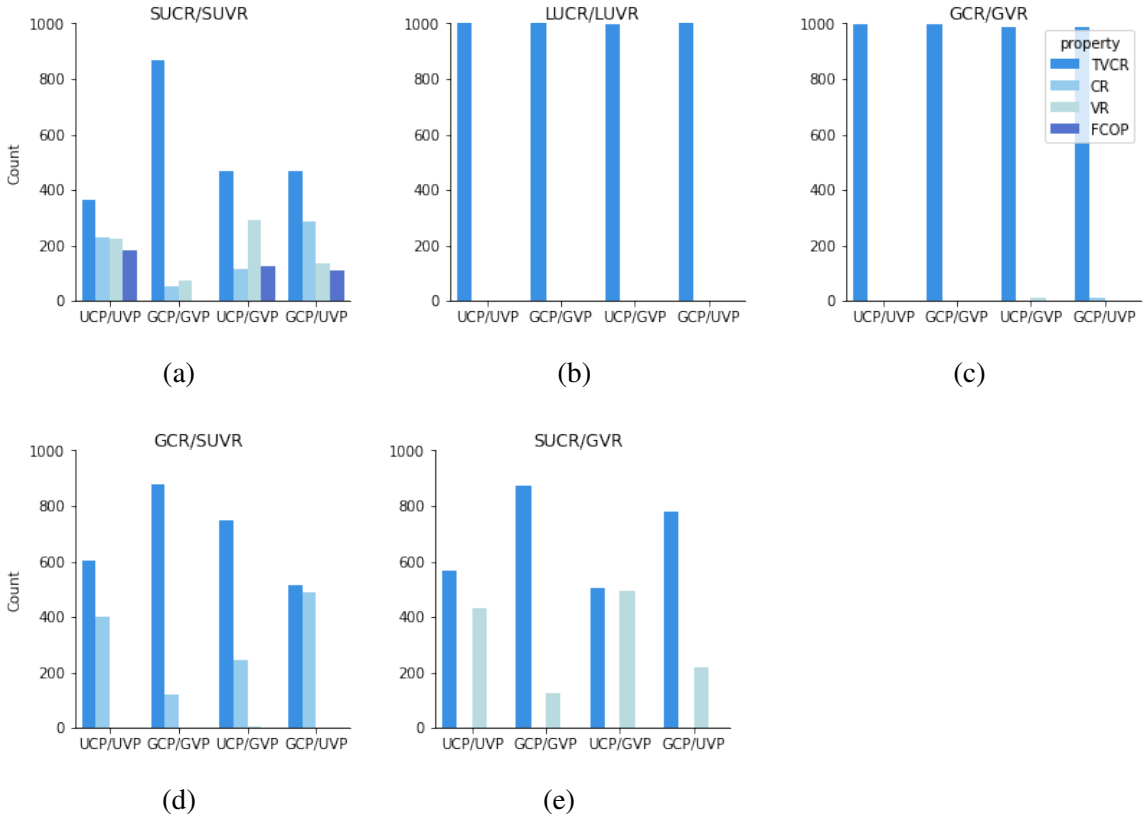


Figure 4.4 Number of profiles with a given property, for the elections of 40 candidates and 40 voters, from Experiment 2. Results are grouped by the parameters used for the profile generation. Each subplot corresponds to the selected radius candidate/voter model, and position candidate/voter models are represented on x-axis.

In the subsequent discussion we will focus mostly on the results for smaller elections from Figure 4.3. We do so because there the differences between the parameters are more pronounced.

Position Choice. Let us analyze Figure 4.3c, which presents histograms for the GCR/GVR radius model and various candidate/voter position models. We consider this model to be our baseline scenario, because it has the most balanced distribution of number of profiles, between the selected preference restrictions. We compare the position models in pairs. Starting with the UCP/UVP and the GCP/GVP models, the latter one has noticeably more TVCR profiles. It can be easily explained by looking at the respective approval matrices in Figure 4.2 (2nd column, 1st and 2nd row). The profiles generated from the GCR/GVR model have higher count of approvals, which in turn increases the likelihood of any irregularities in the approvals block structure, which we directly link to the TVCR property.

Now, turning our attention to the second pair of the position models (still for the same radius model), the UCP/GVP and the GCP/UVP, we see that these are two perfectly symmetrical models. We only explain the first one, and keep in mind that the second one is simply a reversed scenario. Looking at the approval matrices in Figure 4.2 (3rd row), we notice that the blocks of approvals are skewed vertically, which is associated with the VR property, i.e., the approvals form intervals in each row of the matrix. On the contrary, in the second model (Figure 4.2 (4th row)) the blocks are skewed horizontally, which links to the CR property.

Radius Choice. From Figure 4.3, it is clear that the choice of the radius candidate/voter model can significantly reinforce or subside trends in the distribution of numbers of profiles with a selected property. Figure 4.3c was our baseline scenario when analyzing influence of the position candidate/voter models. Now, let us compare this subplot with the other ones. For example, the mixed radius models, that is, the GCR/SUVR (Figure 4.3d) and the SUCR/GVR (Figure 4.3e) models. Looking at the histograms for these models, we observe significant increase in the number of profiles with the CR and the VR properties, for the GCR/SUVR and the SUCR/GVR respectively. The core idea explaining this phenomenon is exactly the same as the one that we already described, when explaining the UCP/GVP and the GCP/UVP position models. Looking at approval matrices in Figure 4.2 (4th and 5th columns), the GCR/SUVR and the SUCR/GVR models output more consistent approval blocks (without 0's between the 1's), resulting in a large decrease of the number of the TVCR profiles. At the same time, these models work in synergy with the UCP/GVP and the GCP/UVP position models, and the created approval structures are skewed either horizontally or vertically, increasing the number of the VR or the CR profiles.

The striking difference between histograms for the LUCR/LUVR (Figure 4.3b) and the SUCR/SUVR (Figure 4.3a) radius models is again related to the inconsistencies in the approval blocks. They are easily observed in approval matrices visualized in Figure 4.2 (1st and 3rd column). Smaller radius range (the SUCR/SUVR) produces much more structured profiles, hence large number of the FCOP profiles. In contrast, large radius range (the LUCR/LUVR) outputs irregular approval blocks in the profiles, increasing the number of the TVCR profiles.

Summary. Most of the conclusions we draw are in line with the basic intuition. The results are very promising, we showed that the VCR domain can be significantly larger than the VR and the CR ones combined. We hope, this will encourage further development and research related to the VCR domain.

4.2 Transforming a TVCR Profile into a CR or a VR One

A natural question that arises from the previous experiments, is how different are the TVCR profiles from the VR or the CR ones? In other words, to what extent do we need to change a TVCR profile for it to become a VR or a CR one. To answer this question, we design yet another experiment.

4.2.1 Experiment 3: How different is a TVCR profile?

In this experiment we partially reuse the strategy for profile generation from the previous one. We narrow down our experiment parameters to only two radius generation methods, that is, the GCR/GVR and the LUCR/LUVR models. Nonetheless, we keep the position models unchanged and explore all four of them. Obviously, we only pick the TVCR profiles. To be exact, we use 400 of them for each category pair (in total, 8 pairs). Due to the limited computational resources, we only use profiles consisting of 15 candidates and 15 voters. Having a TVCR profile as input, we search for a minimal number of agents that we have to remove from this profile, for it to become a VR or a CR one. To achieve more meaningful results we categorize experiment parameters, and describe them below.

Agent Deletion Parameter. We consider three methods to transform a TVCR profile into a VR or a CR one:

1. The *Candidate Deletion (CD)* method, where we search for a minimal number d, such that, after deleting d candidates from the profile it becomes a VR or a CR profile.

2. The *Voter Deletion (VD)* method, which is similar to the CD method, but this time we delete d voters instead.

3. The *Candidate and Voter Deletion (CVD)* method, where we combine the two previous methods, and we simultaneously delete d_c candidates and d_v voters.

Additionally, we put an upper bound on the number of removed agents. For the CD and the VD methods the limit is 4, that is, we only try to remove 4 agents at most. If we fail to find a solution, we use a number 5 as a result, which we interpret as 5 or more agent deletions required to succesfully transform a profile. For the CVD method, the limit is 4 per each agent type, i.e, the maximum number of deleted agents is 4 candidates and 4 voters.

Transformation Result Parameter. Apart from choosing what type of agent (candidate / voter) to delete, for each profile we collect the transformation results in three groups. By the transformation result we mean the lowest possible cost of the transformation, i.e., the minimal number of agent deletions for a TVCR profile to become a VR or a CR one. We collect the following information:

1. The *Candidate Range (CR)*, where the cost is a number of deletions required to get a CR profile.

2. The *Voter Range (VR)*, where the cost is a number of deletions required to get a VR profile.

3. The *Candidate Range or Voter Range (CRVR)*, where the cost is a minimum from the previous two groups.

Results

We divide the observations in two groups based on the agent deletion parameter. First, we analyze the results for the CD and VD methods, and later we discuss the CVD method by itself. We use the similar ideas and concepts as in previous experiments to explain the results.

We present the results for the first group in Figure 4.5. The figure is split into six subplots. Each one of them is a visualization for different subset of results, divided by two parameters. First category is the radius candidate/voter model used to generate the profiles, and second one is based on the transformation result parameter, i.e, the CR, VR and CRVR types. A single subplot shows the distribution of a required number of agents to remove (a violin plot), to transform a TVCR profile into one with a given property.

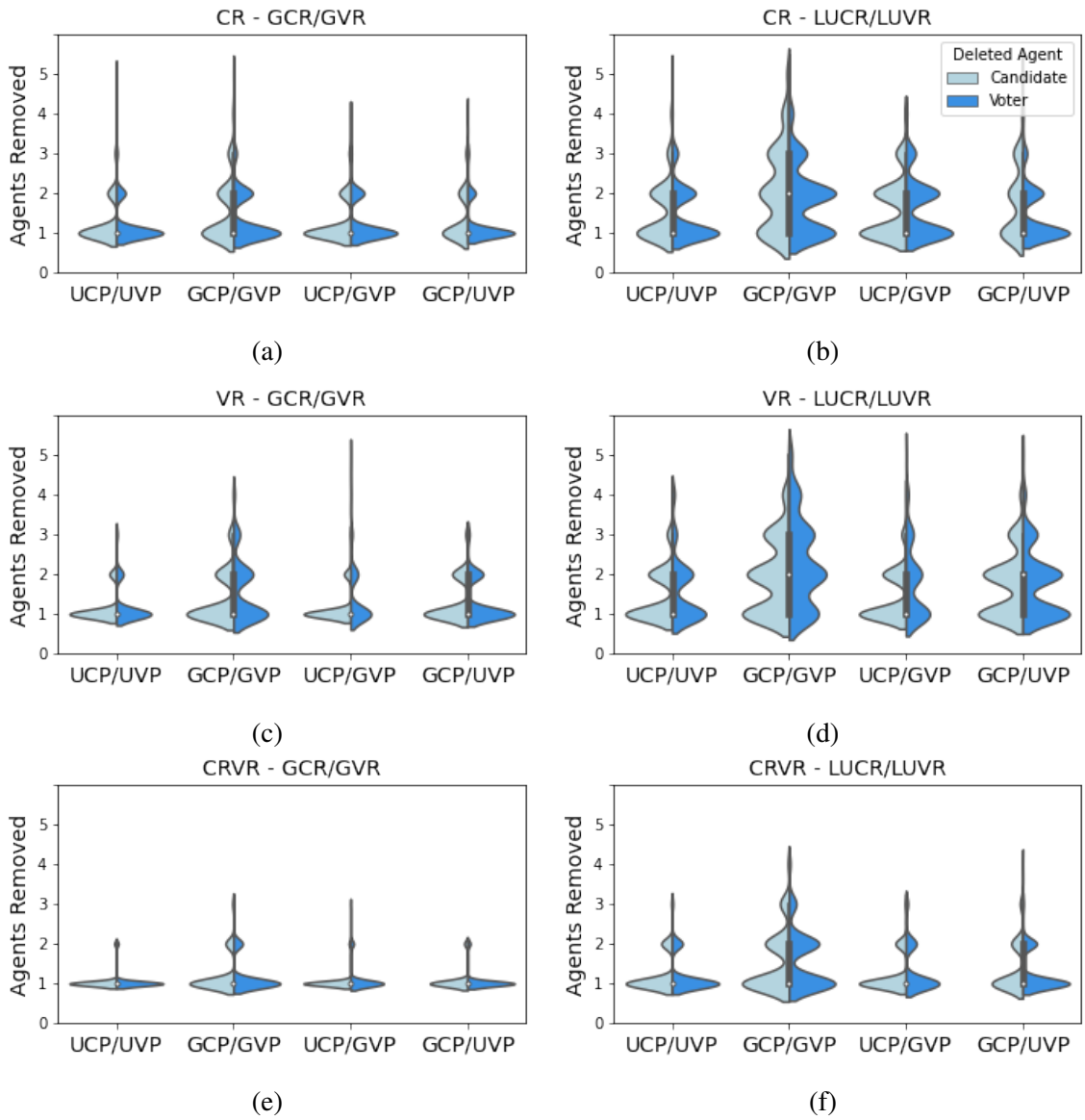


Figure 4.5 Required number of agents to remove, to transform a TVCR profile into a CR or a VR one, from Experiment 3. Each subplot consists of 4 violins, which are similar to a box and whisker plot. A violin plot shows the distribution of quantitative data, and uses kernel density estimation for the visualization. In our case, violins are additionally split in half, i.e., the left part (light color) shows a number of candidates deleted, and the right part (dark color) a number of voters deleted. Results are grouped by the experiment parameters. To create a subplot gird, we use radius candidate/voter models for the columns, and transformation result parameters for the rows, e.g., top-left subplot is a visualization of results for the GCR/GVR radius model and the CR property. Furthermore, position candidate/voter models are represented on x-axis.

Continuing, let us analyze Figure 4.5a and Figure 4.5c. They both show results for the GCR/GVR radius model, the first one corresponds to the CR and the second one to the VR properties. We notice that they are somewhat symmetrical. For example, for the GCP/GVP position model, to obtain the CR profile it is preferable to delete voters. However, if we look at the VR case, it is better to delete candidates. When saying preferred or better, we mean, it requires fewer agent removals. Another interesting observation in these plots, are the violins for the UCP/GVP and the GCP/UVP models. They are virtually swapped between the CR and the VR cases. To explain this, we go back to the respective histograms from Experiment 2 in Figure 4.3c. For the UCP/GVP model, the majority of profiles are VR (not counting the TVCR ones). This suggests that the TVCR profiles from this model are more similar to the VR ones than to the CR ones. This idea is reflected in the corresponding violin plots (Figure 4.5c), it is preferred to delete candidates to obtain the VR property, for profiles from the UCP/GVP position model and the GCR/GVR radius model. For the GCP/UVP position model we see the reversed scenario, in the histograms in Figure 4.3c the majority of profiles are CR, and the violin plots (Figure 4.5a) show that it is better to delete voters to obtain the CR profiles. We observe the same phenomenon for the LUCR/LUVR radius model in Figure 4.5b (the CR) and in Figure 4.5d (the VR). There is nothing unexpected in results for the CRVR property (Figure 4.5e and Figure 4.5f). Having said that, they give us a promising conclusion, that if we are indifferent whether we obtain the CR or the VR profile, we can greatly reduce the required number of agent deletions.

Let us discuss the second part of the experiment, that is results for the CVD method, in which we delete both the candidates and the voters simultaneously. We create plots in Figure 4.6 in the same way as in already described Figure 4.5. Only difference is conceptual, previously violins were split to distinguish between removed agent type, i.e, left half (light color) represented candidates and right half (dark color) showed voters. Now, the halves still represent the number of deleted agents, but process of deletion is simultaneous for candidates and voters. We note that if all the values in distribution are identical, then the violin plot is replaced with a single horizontal line. In most cases, it is enough to delete just one candidate and one voter to obtain a CR or a VR profile. Besides that, we explain why it appears that we need to delete more voters than candidates. It comes merely from implementation nuances. To find a number of required removals, we iterate thorough pairs of all (d_c, d_v) element combinations. That is we start by deleting one candidate and one voter, if there is no solution, then we try to delete one candidate and two voters, and only then if there is still no solution, we try two candidates and one voter, followed by 2 candidates and 2 voters, and so on.

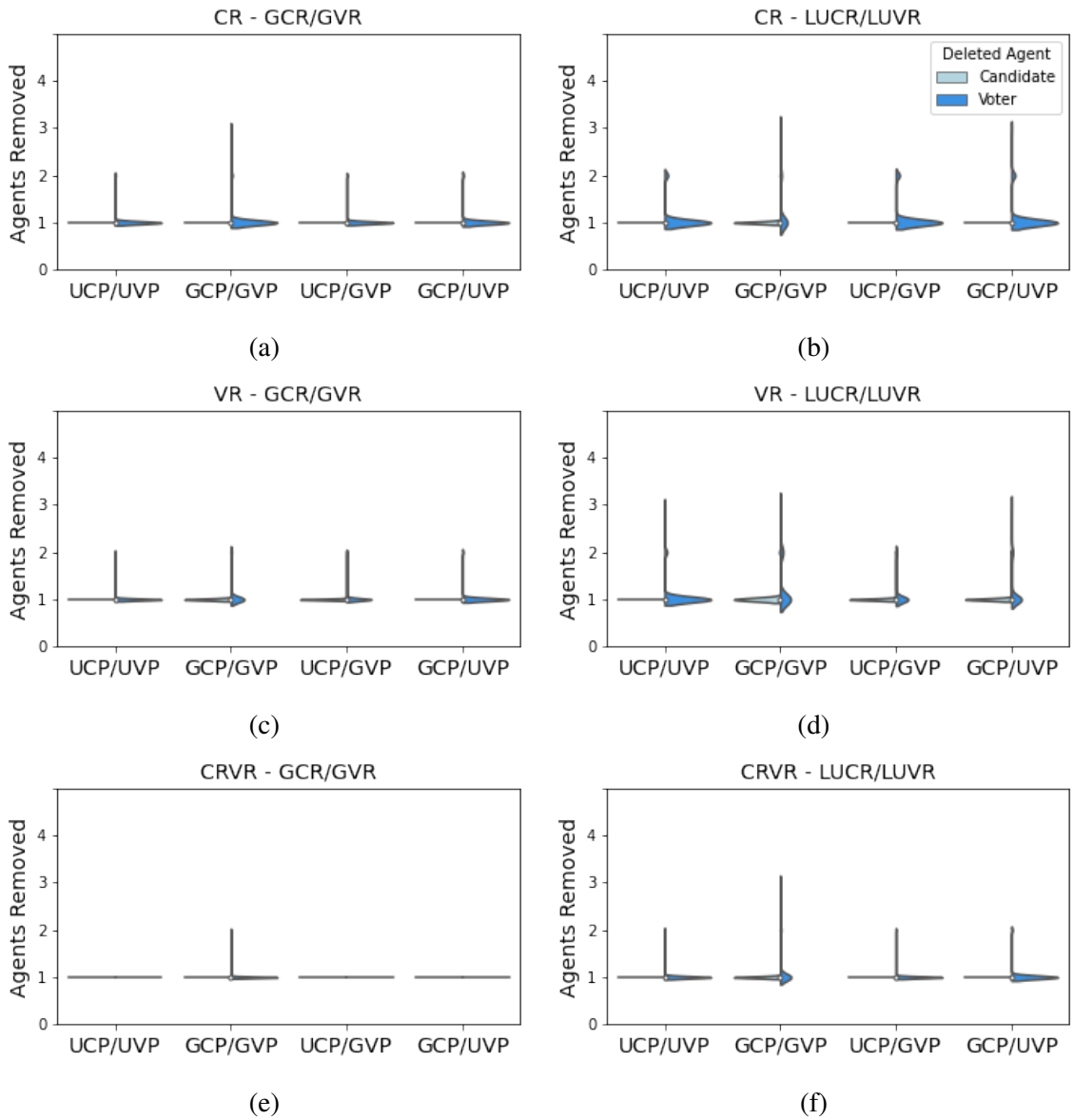


Figure 4.6 Required number of candidates and voters to remove, to transform a TVCR profile into a CR or a VR one, from Experiment 3. The plot is structured in the same way as in Figure 4.5, except one difference. Split in half violins no longer represent, independent deletion of the candidates or the voters, but correspond to simultaneous deletion of both, the candidates and the voters. If there is no violin (horizontal line), it means that all the values were identical.

In addition to the visual representation of the results we provide statistical metrics of the results, i.e., mean, standard deviation, minimum and maximum. Values are grouped by experiment parameters, in the same way as in the graphs, except the data is aggregated for all

the position models. We present them in Table 4.1 for the first part of the experiment, and in Table 4.2 for the second one.

Radius Model	Property	Deleted Agent	mean	std	min	max
GCR/GVR	CR	Candidate	1.366	0.633	1.000	5.000
		Voter	1.258	0.506	1.000	4.000
	CRVR	Candidate	1.089	0.304	1.000	3.000
		Voter	1.092	0.297	1.000	3.000
	VR	Candidate	1.256	0.513	1.000	4.000
		Voter	1.373	0.623	1.000	5.000
LUCR/LUVR	CR	Candidate	1.788	0.903	1.000	5.000
		Voter	1.587	0.737	1.000	5.000
	CRVR	Candidate	1.341	0.593	1.000	4.000
		Voter	1.336	0.567	1.000	4.000
	VR	Candidate	1.656	0.792	1.000	5.000
		Voter	1.851	0.935	1.000	5.000

Table 4.1 Statistical metrics for number of deleted agents, from Experiment 3.1

Radius Model	Property	Deleted Agent	mean	std	min	max
GCR/GVR	CR	Candidate	1.000	0.000	1.000	1.000
		Voter	1.021	0.146	1.000	3.000
	CRVR	Candidate	1.000	0.000	1.000	1.000
		Voter	1.001	0.025	1.000	2.000
	VR	Candidate	1.002	0.043	1.000	2.000
		Voter	1.022	0.148	1.000	2.000
LUCR/LUVR	CR	Candidate	1.002	0.043	1.000	2.000
		Voter	1.107	0.332	1.000	3.000
	CRVR	Candidate	1.001	0.035	1.000	2.000
		Voter	1.029	0.171	1.000	3.000
	VR	Candidate	1.008	0.090	1.000	2.000
		Voter	1.127	0.357	1.000	3.000

Table 4.2 Statistical metrics for number of deleted agents, from Experiment 3.2

Summary. First, we showed how results from Experiment 3 further confirm our conclusions from Experiment 2. Moreover, our results provide strong basis, to consider transforming a TVCR profile into a CR or a VR one, if it would be required by other applications.

Chapter 5

Control Problem

5.1 Overview

5.1.1 AV-CC-DV for Single Peaked (VR = CI = row C1P = $r_c = 0$)

Algorithm from Faliszewski et al. (2009)

1. kandydat p na osi, pozostali na lewo i prawo
2. wybór przeciwników (groźny kandydat):
   (a) pierwszy na lewo c_L i pierwszy na prawo c_R takich, że $score(c_?) \geq score(p)$
   (b) powtarzamy dla kolejnych lewych c_{L_i} takich, że $score(c_{L_i}) > score(c_{L_{i-1}})$
   (c) analogicznie dla prawych c_{R_i} takich, że $score(c_{R_i}) > score(c_{R_{i-1}})$
3. usuwanie głosów - zaczynając od najbliższego prawego przeciwnika c_R (dla lewych - odbicie lustrzane)
   (a) notacja v: $b(v)$ początek przedziału (lewy) i $e(v)$ koniec przedziały (prawy)
   (b) wybieramy głosujących $\widehat{V}$ takich że $\{v \in V | \, b(v) \in (p, c_R]\}$
   (c) sortujemy $\widehat{V}$ malejąco po $e(v)$ (od najbardziej prawego końca przedziału)
   (d) usuwamy $scoreDelta(p, c_R) + 1$ głosów
4. powtarzamy do skutu, przeliczając aprobaty co iterację

5.1.2 AV-CC-DV for Candidate Range = VI = column C1P = $r_v = 0$

Algorithm from Magiera and Faliszewski (2014)

1. notacja CR dla c: $v_{f(c)}$ początek przedziału (lewy/górny) i $v_{e(c)}$ koniec przedziału (prawy/dolny)
2. kandydat p
3. Głosujący - wybieramy głosujących $V' = \{v \in V | \, p \notin A(v)\}$
4. Przeciwnicy - wybieramy kandydatów $\widehat{C} = \{c \in C \setminus \{p\} | \, scoreDelta(c, p) \geq 0\}$
5. Usuwanie głosów
   (a) zaczynając od przeciwnika c_N takiego, że $v_{e(c_N)}$ jest minimalne
   (b) usuwamy $deltaScore(c_N, p) + 1$ głosów, zaczynając od końca przedziału $v_{e(c_N)}$
6. powtarzamy do skutu, przeliczając V' i $\widehat{C}$ co iterację

5.2 AV-CC-DV for the VCR profiles

Notation used in the algorithm and later in the proofs:
x_a, r_a — point and radius
$b(a) = x_a - r_a$ — range beginning (left)
$e(a) = x_a + r_a$ — range end (right)
p — candidate chosen to be a winner
$\widehat{C} = \{c \in C \setminus \{p\} | \, scoreDelta(c, p) \geq 0\}$ — opponents (dangerous candidates)
$V' = \{v \in V | \, p \notin A(v)\}$ — potential voters to delete

We use “smart” greedy strategy for voter deletion:

1. We pick a leftmost opponent c_L that is the one with the minimal $b(c_L)$
2. We consider only voters who approve of c_L that is, $\widehat{V} = \{v \in V' | \, c_L \in A(v)\}$
3. We need to delete $deltaScore(c_L, p) + 1$ rightmost voters from $\widehat{V}$ that is the ones with the maximal $e(v)$, if it’s not possible we fail
4. We repeat the above steps until we succeed, recalculating $\widehat{C}$ and $\widehat{V}$ after each iteration

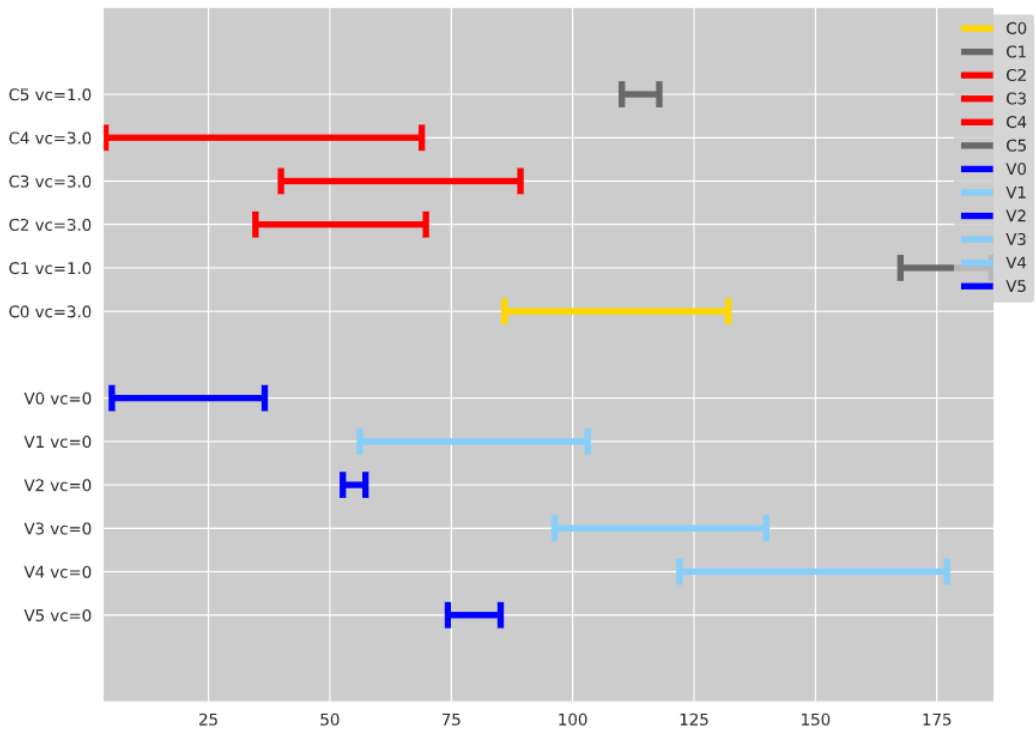


Figure 5.1 Temporary Image - VCR !C1P Election
yellow — candidate p
red — opponents
lightblue — voters whitelist
darkblue — possible voters to delete
grey — agents that can be ignored

5.2.1 Sketch Proof - CR case

We will show how the VCR algorithm is equivalent to the CR one from Magiera and Faliszewski (2014). Having voters in CR order, that is, they are still represented by a point x_v on the $\mathbb{R}$ line but have radius 0. Example in Figure 5.2.

Choosing opponents

In the CR algorithm, we choose candidates to process, ordered by $v_{e(c)}$ ascending. We can see that it is equivalent to picking the leftmost $(min(b(c)))$ candidate from the set of opponents $\widehat{C}$ in the VCR algorithm, because it will also have the minimal $v_{e(c)}$.

There is a special case of "contained candidate", for which the above statement is not true. In Section 5.2.3, we explain in detail why it is irrelevant for the overall correctness of the algorithm.

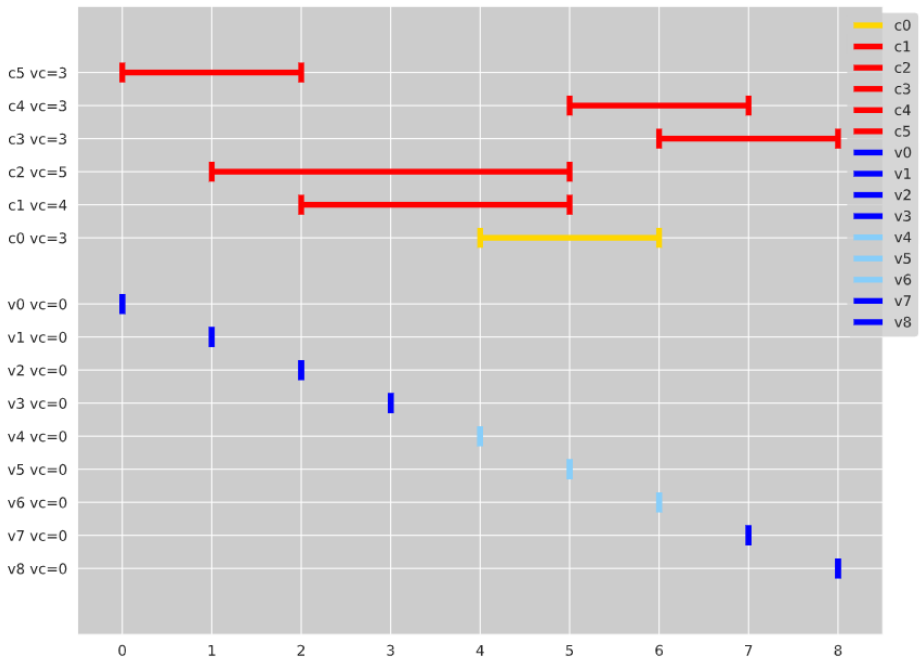


Figure 5.2 Temporary Image - CR Election

Voter deletion

Now, once we now how to pick an opponent c_L, we will show that choosing voters is equivalent as well. Notion of the beginning/end of the interval in the CR order is equivalent to the left/right end of the range in the VCR setting. Both algorithms try to delete $scoreDelta(p, c_L) + 1$ voters, CR one starts deleting voters from the end of interval, while VCR does virtually the same by deleting the rightmost voter first ($max(e(v))$).

5.2.2 Sketch Proof - VR case

We will show how the VCR algorithm is equivalent to the VR one from Faliszewski et al. (2009). Having candidates in VR order, that is, they are still represented by a point x_c on the $\mathbb{R}$ line but have radius 0. Example in Figure 5.3.

Choosing opponents

VR algorithm picks dangerous candidates to the left and right of candidate p, however later it proceeds only with defeating the right ones. Once finished with them, it does the "mirror flip" of the universe to process candidates on the left side. Even though the VCR algorithm picks leftmost candidate, we will show it is equivalent to the VR strategy under some basic assumptions.

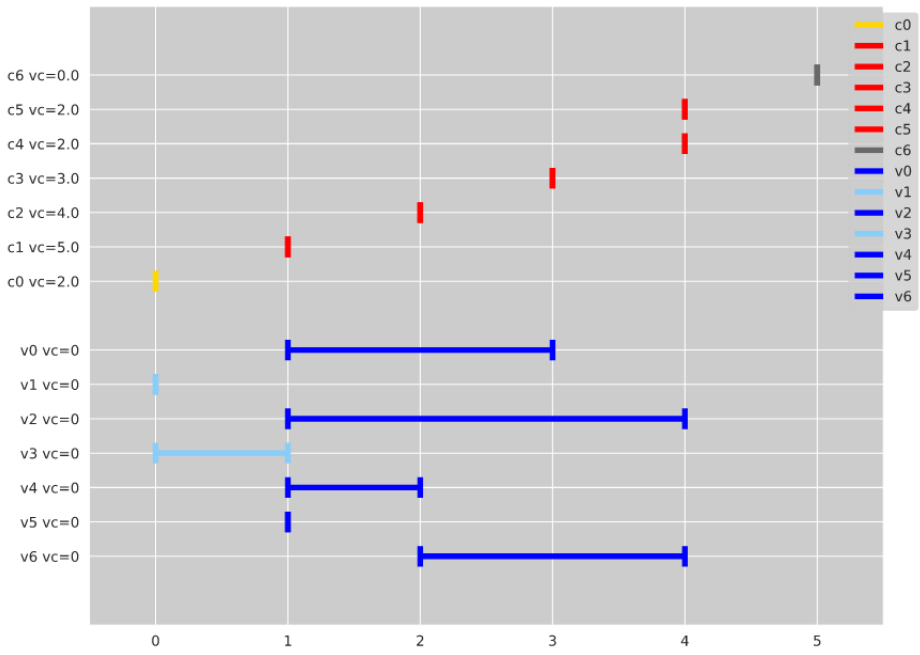


Figure 5.3 Temporary Image - VR Election

Following example in Figure 5.3, let's assume that candidate p always will be the first candidate in the VR order, which is equivalent to being the leftmost in the VCR order. Then we can see that the VCR algorithm strategy of choosing the leftmost candidate as an opponent will be the same as the VR algorithm choosing the first candidate on the right side of p. In the example it would be candidate c_1.

This assumption is correct because, having candidates in order $[c_L, p, c_R]$, if there was a voter v who approved of c_L and c_R it would also have to approve of p (because of the VR property), so we could not delete v anyway.

Now, expanding on the above, ...

(WIP: Faktycznie taki dowód nie jest w 100% przekonujący Mam 2 możliwości: 1) Wprowadzić krork "universe flip" do algorytmu i wtedy na pewno działa. 2) Zmienić argumentację i pokazać, że wybieranie najdalszego kandydat od p (najbardziej lewy po lewej stronie) też jest w porządku)

Voter deletion

The reasoning here is very similar to the one in the voter deletion part of the CR proof. Both algorithms try to delete $scoreDelta(c, p) + 1$ voters. VR algorithm starts deleting voters from the end of interval $(max(e(v)))$. VCR one does virtually the same by deleting the rightmost voter first.

5.2.3 Sketch Proof - VCR (not C1P) case

Definition 5.2.1. *We introduce an alternative representation of an agent in a VCR profile. Every agent a is represented by the point x_a and the radius r_a. We define a's range beginning as $b(a) = x_a - r_a$, and a's range end as $e(a) = x_a + r_a$.*

Definition 5.2.2. *Having a solution S to the AV-CC-VD problem, which deletes k voters. We define the score of this solution as,*

$$score(S) = \sum_{i=1}^{k} e(v_i)$$

(WIP: Dlaczego możemy zdefiniować jakiś wynik, czy wynik jest po prostu równy k?)

Proposition 3. *VCR-AV-CC-VD finds an optimal solution.*

Proof. Let S be the solution found by the VCR-AV-CC-VD algorithm, and S^* an optimal solution to the problem. We will prove that $S = S^*$ by contradiction. Assume $S \neq S^*$, therefore, $S - S^* \neq \emptyset$. Let v, be any voter in $S - S^*$. When v was added to S, it was the rightmost voter ($\max(e(v))$). Since S^* is optimal, then there must be some voter $x \in S^*$, such that $e(x) < e(v)$, because v is the rightmost voter. Let $S^{*\prime} = S^* \cup \{v\} - \{x\}$, we know that $S^{*\prime}$ is a correct solution. (WIP: Dlaczego $S^{*\prime}$ jest poprawny? - Czy to właśnie tu musimy uzasadnić specjalny przpadek "wewnętrznych kandydatów"? Czy może być to jakiś osobny, wcześniej pokazany lemat?) However,

$$score(S^{*\prime}) = score(S^*) + e(v) - e(x) > score(S^*)$$

contradicting that S^* is an optimal solution. We have reached a contradiction, so our assumption must have been wrong. Thus $S = S^*$, hence S is an optimal solution. □

Special case - contained candidates - alternative explanation

Now we will show why our solution is correct in the general VCR setting, even when the profile does not have VR/CR property, example in Figure 5.1. The proof is split into two stages, similarly to the previous ones. But first we will explain one special case of the election, that is worth examining.

In the VCR model it is possible that some candidates have their interval fully contained in the other's interval. Every voter who would approve the smaller candidate would also approve the larger one, however not vice versa. For this case, our strategy of choosing voters to delete might seem suboptimal, at first glance.

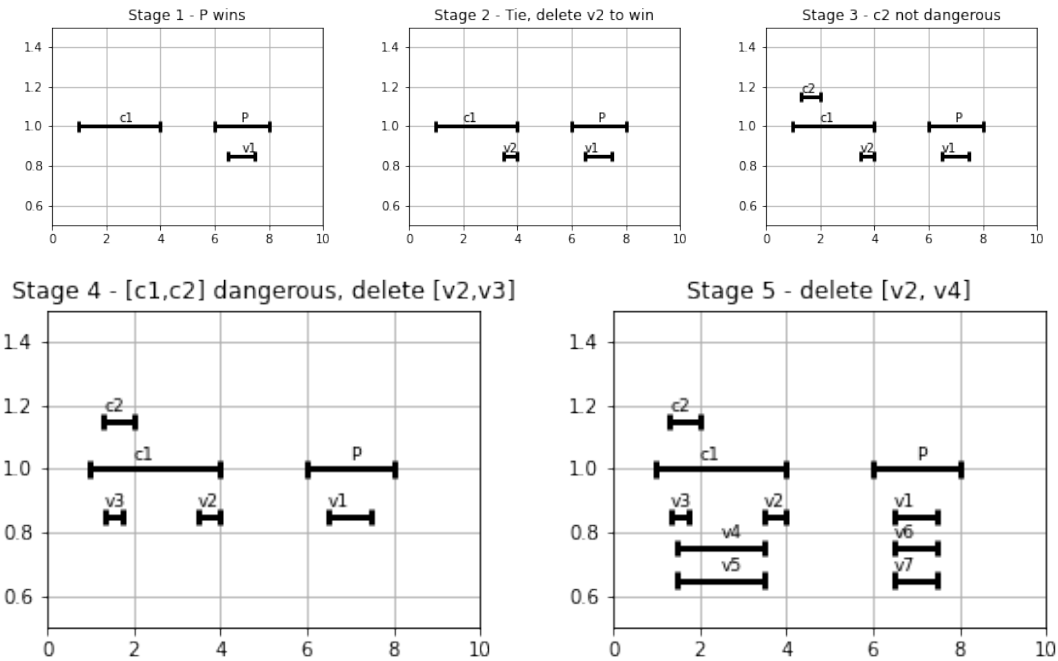


Figure 5.4 Temporary Image - Special Case

We explain the issue, starting with the example of the most basic election in Figure 5.4. Stages 1 through 3 are trivial, in the subsequent stages we make the election more and more complex by adding new agents. Turning our attention to Stage 4, now both candidates c_1 and c_2 are dangerous, with scores 2 and 1 respectively. Candidate p needs to defeat c_2 by one vote, and c_1 by two votes. In Stage 5 we add multiple more voters ...

The core idea to solve our issue is understanding what types of the new votes we can add. Let's consider only the voters who approve of c_1 and/or c_2.

1. A voter who approves only c_1 and not c_2, in this case, $scoreDelta(p, c_1)$ would increase by 1, and $scoreDelta(p, c_2)$ would stay the same.

2. A voter who approves both c_1 and c_2, this time, $scoreDelta(p, c_1)$ and $scoreDelta(p, c_2)$ would increase equally by 1.

Proposition 4. *Having external candidate c_E and another internal one c_I, which is fully contained inside external's range.*

Let:

1. $K' = \{v \in \widehat{V} \mid v \in A(c_E) \wedge v \notin A(c_I)\}$.

2. $|K'| = scoreDelta(p, c_E) - scoreDelta(p, c_I)$.

3. *$d(c) = score(c) - score(p) + 1$, i.e., how many voters we need to delete for p to defeat c.*

Then $|K'| \geq 0$ is always true, and the size of set K' will be equal to the $d(c_E) - d(c_I)$

Proof. tbc. □

Special case - contained candidates

In the VCR model it is possible that some candidates have their interval fully contained in the other's interval. Every voter who would approve the smaller candidate would also approve the larger one, however not vice versa. For this case, our strategy of choosing voters to delete might seem suboptimal, at first glance.

The example from Figure 5.5 illustrates why we do not have to worry about this special case. Every contained voter will have as much or less total votes than the larger candidate, he/she might not even be a dangerous candidate at all. Now we see that in the process of defeating the larger candidate we will automatically defeat the smaller one.

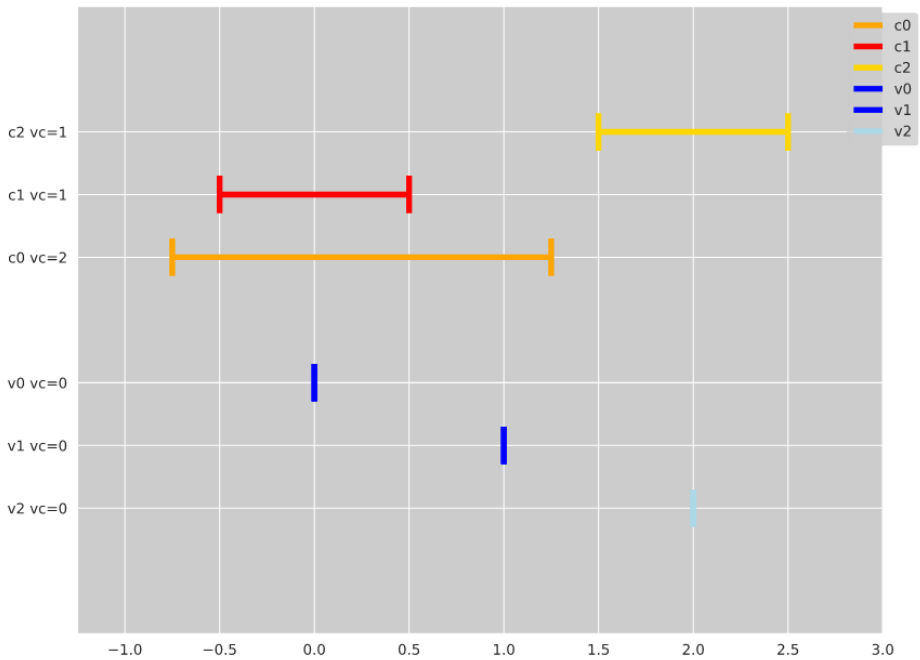


Figure 5.5 Temporary Image - Special Case

Choosing opponents

We consider only dangerous candidates (opponents), that is the ones with score higher or equal to the score of p. We process opponents one by one, starting with the one who has the

leftmost beginning of the range, i.e., $min(b(c))$. After each iteration, we recompute the set of dangerous candidates. Thanks to this we are sure that all the steps we perform are necessary to find a solution (if it exists), hence optimal.

Voter deletion

Having currently processed opponent c_L, we consider only the voters who approve of him/her and do not approve of p, that is the voters whose range beginning is inside the candidate's range $\{v \in V' \mid b(v) \in [b(c_L), e(c_L)]\}$. We need to delete $scoreDelta(c_L, p) + 1$ voters to defeat the opponent. By picking the voters in order of the rightmost end of the range, i.e., $v_R = max(e(v))$, we try to maximize the number of other candidates who might be in the range $[b(v_R), e(v_R)]$. We don't have to worry about any candidate to the left, because of our leftmost candidate picking strategy, they are already defeated. This strategy ensures the solution is optimal if it exists.

5.3 AV-CC-AV for the VCR profiles

Our strategy is based on the solution from Magiera and Faliszewski (2014).

1. Add all $v \in W$ to the election
2. Is p the winner?
3. Try to delete $k' = |W| - k$ voters

Chapter 6

Summary

6.1 Conclusions

6.2 Future Work

Bibliography

Arrow, K. (1951). Social Choice and Individual Values. Wiley: New York.

Aziz, H. and Walsh, T. (2014). Justified representation in approval-based committee voting. CoRR, abs/1407.8269.

Barberà, S. and Moreno, B. (2011). Top monotonicity: A common root for single peakedness, single crossing and the median voter result. Games and Economic Behavior, 73(2):345–359.

Bartholdi, J. J., Tovey, C. A., and Trick, M. A. (1989). The computational difficulty of manipulating an election. Social Choice and Welfare, 6(3):227–241.

Bartholdi, J. J., Tovey, C. A., and Trick, M. A. (1992). How hard is it to control an election? Mathematical and Computer Modelling, 16(8):27–40.

Black, D. (1958). The theory of committees and elections. University Press, Cambridge.

Booth, K. S. and Lueker, G. S. (1976). Testing for the consecutive ones property, interval graphs, and graph planarity using pq-tree algorithms. Journal of Computer and System Sciences, 13(3):335 – 379.

Brams, S. and Fishburn, P. (1983). Approval Voting. Southgate Publishers.

Brandt, F., Brill, M., Hemaspaandra, E., and Hemaspaandra, L. A. (2015). Bypassing combinatorial protections: Polynomial-time algorithms for single-peaked electorates. Journal of Artificial Intelligence Research, 53:439–496.

Brandt, F., Conitzer, V., Endriss, U., Lang, J., and Procaccia, A. D., editors (2016). Handbook of Computational Social Choice. Cambridge University Press.

Elkind, E., Faliszewski, P., and Skowron, P. (2020). A characterization of the single-peaked single-crossing domain. Social Choice and Welfare, 54(1):167–181.

Elkind, E., Faliszewski, P., Skowron, P., and Slinko, A. (2017). Properties of multiwinner voting rules. Soc. Choice Welf., 48(3):599–632.

Elkind, E. and Lackner, M. (2015). Structure in dichotomous preferences. CoRR, abs/1505.00341.

Elkind, E., Lackner, M., and Peters, D. (2016). Preference restrictions in computational social choice: Recent progress. In Proceedings of the Twenty-Fifth International Joint Conference on Artificial Intelligence, IJCAI'16, page 4062–4065. AAAI Press.

Faliszewski, P., Hemaspaandra, E., and Hemaspaandra, L. A. (2006). How hard is bribery in elections? CoRR, abs/cs/0608081.

Faliszewski, P., Hemaspaandra, E., Hemaspaandra, L. A., and Rothe, J. (2009). The shield that never was: Societies with single-peaked preferences are more open to manipulation and control. CoRR, abs/0909.3257.

Faliszewski, P. and Procaccia, A. D. (2010). Ai's war on manipulation: Are we winning? AI Magazine, 31(4):53–64.

Faliszewski, P., Skowron, P., Slink, A., and Talmon, N. (2017). Multiwinner Voting: A New Challenge for Social Choice Theory. AI Access.

Fitzsimmons, Z. and Lackner, M. (2019). Incomplete preferences in single-peaked electorates. CoRR, abs/1907.00752.

Godziszewski, M., Batko, P., Skowron, P., and Faliszewski, P. (2021). An analysis of approval-based committee rules for 2d-euclidean elections.

Gurobi Optimization, L. (2021). Gurobi optimizer reference manual.

Hemaspaandra, E., Hemaspaandra, L. A., and Rothe, J. (2009). Hybrid elections broaden complexity-theoretic resistance to control. Mathematical Logic Quarterly, 55(4):397–424.

Lackner, M. and Skowron, P. (2020). Approval-based committee voting: Axioms, algorithms, and applications.

Magiera, K. and Faliszewski, P. (2014). How hard is control in single-crossing elections? Frontiers in Artificial Intelligence and Applications, 263:579–584.

Magiera, K. and Faliszewski, P. (2019). Recognizing top-monotonic preference profiles in polynomial time. J. Artif. Intell. Res., 66:57–84.

Mirrlees, J. A. (1971). An Exploration in the Theory of Optimum Income Taxation. Review of Economic Studies, 38(2):175–208.

Peters, D. (2017). Recognising multidimensional euclidean preferences. In Proceedings of the Thirty-First AAAI Conference on Artificial Intelligence, AAAI'17, page 642–648. AAAI Press.

Peters, D. and Lackner, M. (2020). Preferences single-peaked on a circle. J. Artif. Intell. Res., 68:463–502.

Roberts, K. W. (1977). Voting over income tax schedules. Journal of Public Economics, 8(3):329–340.

Skowron, P., Yu, L., Faliszewski, P., and Elkind, E. (2015). The complexity of fully proportional representation for single-crossing electorates. Theoretical Computer Science, 569:43–57.

Walsh, T. (2007). Uncertainty in preference elicitation and aggregation. In Proceedings of the 22nd National Conference on Artificial Intelligence - Volume 1, AAAI'07, page 3–8. AAAI Press.

Wolsey, L. and Nemhauser, G. (2014). Integer and Combinatorial Optimization. Wiley Series in Discrete Mathematics and Optimization. Wiley.